Vulnerability Assessment Report for a LiveWebsite

Prepared by:	Omar Sawalmeh
Position:	Cyber Security Intern
Date:	4 April 2026
Organization:	Future Interns

CONFIDENTIAL SECURITY ASSESSMENT

Executive Summary

This report presentsthe results of a vulnerability assessment conducted on two live web applications: http://scanme.nmap.org/ and http://testaspnet.vulnweb.com/login.aspx. The assessment aimed to identify potential security weaknesses that could be exploited by attackers and to recommend appropriate remediation measures.

Three key vulnerabilities were identified during the assessment:

1. Missing Anti-Clickjacking Header
2. Absence of Anti-CSRF Tokens
3. Cookie Without SameSite Attribute

Each finding has been analyzed in terms of technical details, business impact, and recommended remediation steps. The vulnerabilities identified are of low to medium risk and can be mitigated through proper configuration and secure coding practices.

Additionally, the assessment included network-level scanning using Nmap to identify exposed services and evaluate the attack surface of the target system. The results showed that only essential services were exposed, indicating a basic level of network security.

Overall, while no critical vulnerabilities were identified, the presence of multiple medium-risk issues highlights the need for improved security configurations and secure development practices. Addressing these findings will reduce the risk of common web-based attacks and enhance overall system resilience.

Table of Contents

1. Executive Summary 1
2. Table of Contents 2
3. List of Figures 3
4. Scope 4
5. Methodology 5
6. Network Scan Results (Nmap) 5
7. Findings 6
7.1 Finding #1: Missing Anti-Clickjacking Header 6
7.2 Finding #2: Absence of Anti-CSRF Tokens 8
7.3 Finding #3: Cookie Without SameSite Attribute 10
7.4 Finding #4: HTTP Only Site (No HTTPS) 11
8. Conclusion 12

List of Figures

Figure 1: Nmap scan results showing open ports and services 5
Figure 2: OWASP ZAP alert showing missing anti-clickjacking header..... 6
Figure 3: HTTP response headers showing absence of X-Frame-Options header..... 7
Figure 4: OWASP ZAP alert showing absence of anti-CSRF tokens 8
Figure 5: Form submission request showing absence of CSRF token 9
Figure 6: Session cookie without SameSite attribute 10
Figure 7: OWASP ZAP alert showing HTTP-only site without HTTPS enforcement 11
Figure 8: Browser security warning indicating page is not secure 11

Scope

Thescopeofthis assessment included:

- Performing a non-intrusive vulnerability scan on the target websites.
- Identifying common web application vulnerabilities related to headers, cookies, and form security.
- Providing actionable recommendations to improve the security posture of the websites.

The assessment did not include penetration testing, denial-of-service testing, or exploitation of vulnerabilities.

Methodology

Thefollowingtoolsandtechniques were used during the assessment:

Nmap: Used to perform network scanning and identify open ports and services on the target host (http://scanme.nmap.org/).

OWASP ZAP (Zed Attack Proxy): Used to perform web application scanning and identify vulnerabilities such as missing headers, insecure cookies, and CSRF weaknesses on (http://testaspnet.vulnweb.com/login.aspx).

Browser Developer Tools: Used to inspect HTTP headers, cookies, and client-side security issues such as missing security headers and insecure configurations.

The methodology followed standard vulnerability assessment practices:

1. Information Gathering
2. Vulnerability Scanning
3. Analysis and Verification
4. Reporting and Recommendations

Network Scan Results (Nmap)

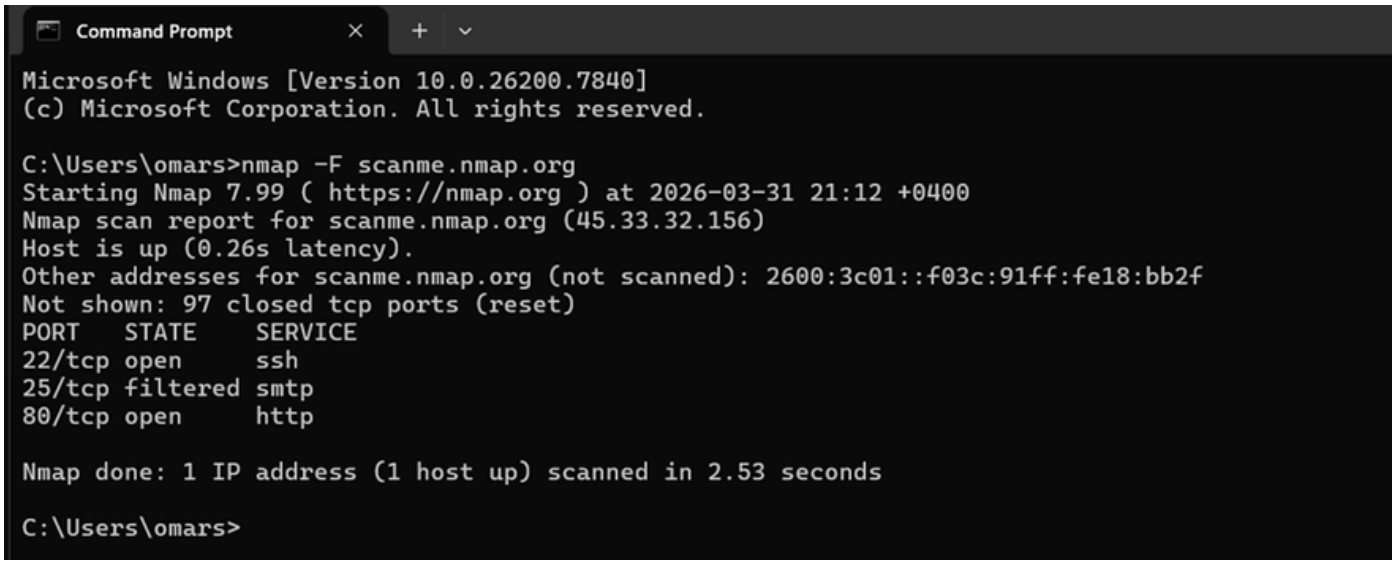
Anetwork scan wasperformedonthetargethost(http://scanme.nmap.org/) using Nmap to identify open ports and running services.

The scan results showed:

- Host is active and reachable
- Open Ports:
  - Port 22 (SSH) – Open
  - Port 80 (HTTP) – Open
  - Port 25 (SMTP) – Filtered

These results indicate that the server is running a web service and allows remote access via SSH. Filtered ports suggest that some security controls such as firewalls are in place.

Figure 1: Nmap scan results showing open ports and services



Findings

Finding #1: Missing Anti-Clickjacking Header

Title	Missing Anti-Clickjacking Header
Risk Level	Medium
Technical Description	The web application does not implement the X-Frame-Options security header. This omission allows the website to be embedded within an iframe on another domain, making it susceptible to clickjacking attacks.
Business Impact	Without this protection, attackers could embed the website within a malicious page and trick users into performing unintended actions, such as clicking hidden buttons or submitting sensitive information. This could lead to unauthorized transactions or data exposure.
Remediation Steps	1. Configure the web server to send the X-Frame-Options header (e.g., DENY or SAMEORIGIN). 2. Implement the Content-Security-Policy (CSP) frame-ancestors directive to control which domains can embed the site. 3. Test the implementation across all pages to ensure compatibility. 4. Verify the presence of the header using browser developer tools.
Timeline	1–2 Days
Effort	Low
Evidence	This issue was identified using OWASP ZAP during passive scanning. This vulnerability was identified using OWASP ZAP and verified using Browser Developer Tools by analyzing the HTTP response headers. The response headers do not include the X-Frame-Options or Content-Security-Policy frame-ancestors directive, confirming the absence of clickjacking protection.

Figure 2: OWASP ZAP alert showing missing anti-clickjacking header

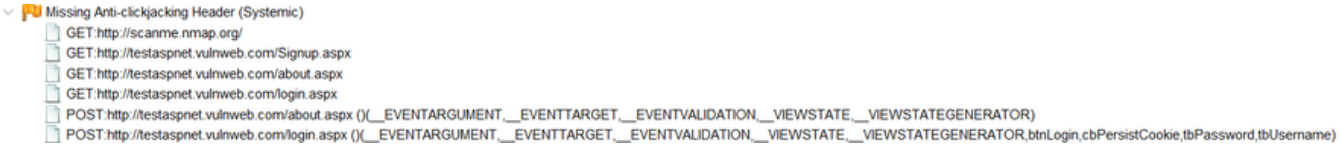
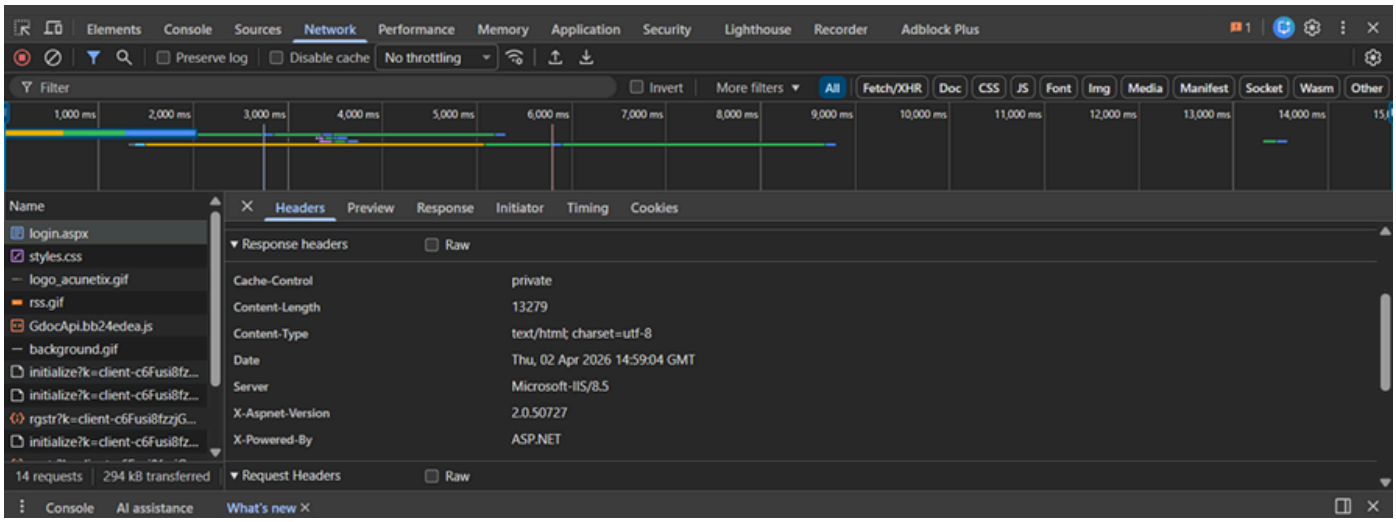


Figure 3: HTTP response headers showing absence of X-Frame-Options header



Finding #2: Absence of Anti-CSRF Tokens

Title	Absence of Anti-CSRF Tokens
Risk Level	Medium
Technical Description	The web application's forms do not include Anti-Cross-Site Request Forgery (CSRF) tokens. This leaves state-changing requests (such as POST requests) vulnerable to being forged by attackers.
Business Impact	An attacker could exploit this weakness by tricking an authenticated user into performing unintended actions, such as changing account details or submitting unauthorized requests. This could compromise user accounts and damage trust in the application.
Remediation Steps	1. Generate a unique, cryptographically strong CSRF token for each user session. 2. Include this token as a hidden field in all HTML forms. 3. Configure the server to reject any request with a missing or invalid token. 4. Enforce SameSite cookie attributes (see Finding #3) as an additional layer of defense.
Timeline	2–3 Days
Effort	Medium
Evidence	This issue was identified using OWASP ZAP and further analyzed using Browser Developer Tools. Inspection of the request payload shows that form submissions do not include any CSRF token, confirming that the application does not implement CSRF protection.

Figure 4: OWASP ZAP alert showing absence of anti-CSRF tokens

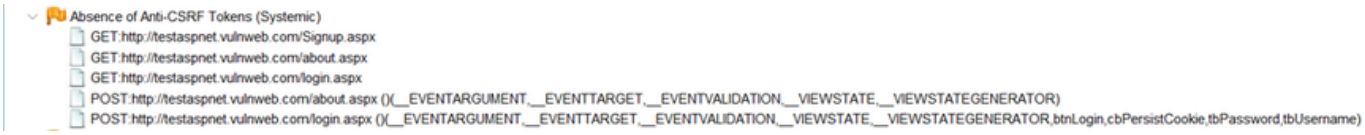
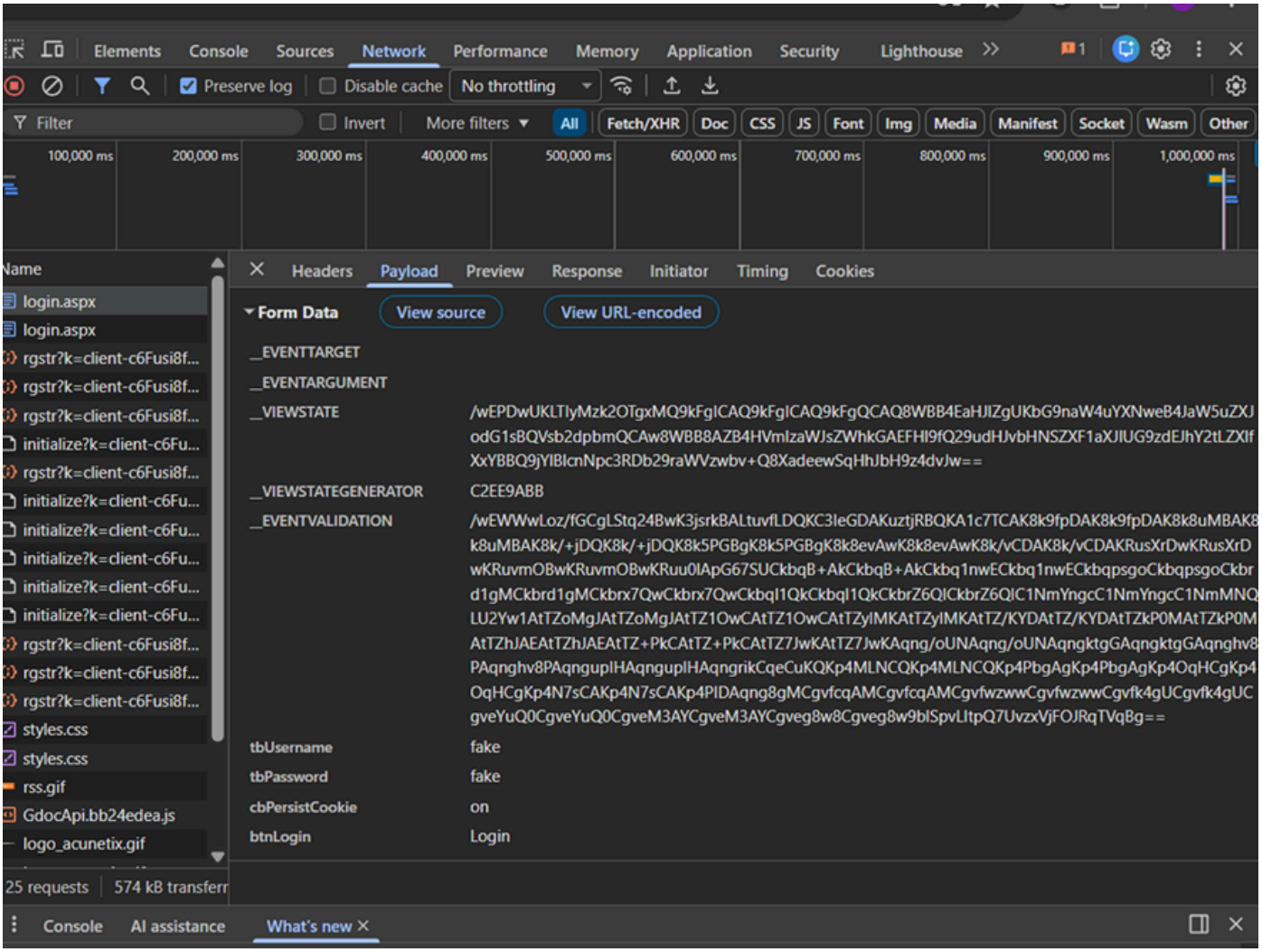


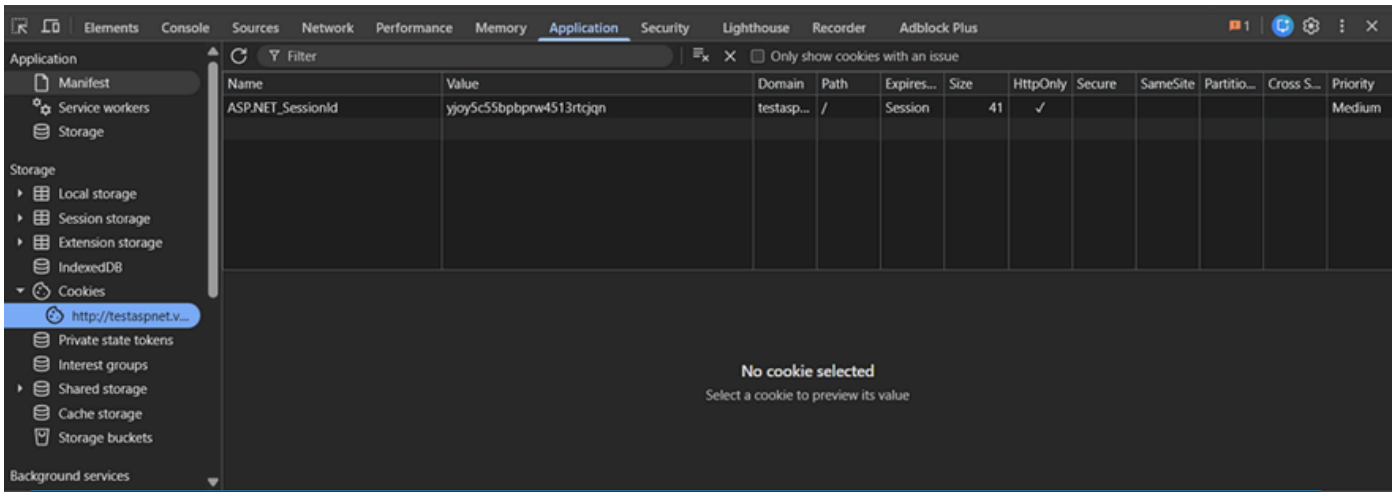
Figure 5: Form submission request showing absence of CSRF token



Finding #3: Cookie Without SameSite Attribute

Title	Cookie Without SameSite Attribute
Risk Level	Low
Technical Description	Session cookies are configured without the SameSite attribute. This configuration allows browsers to send cookies with cross-site requests, potentially enabling CSRF attacks.
Business Impact	Without the SameSite attribute, session cookies may be sent to unintended domains, increasing the risk of session hijacking or unauthorized actions.
Remediation Steps	1. Add SameSite=Strict to all session cookies to prevent cross-site requests. 2. Use SameSite=Lax if limited cross-site navigation is required. 3. Update cookie configuration in the application framework settings. 4. Test authentication and session management flows after implementation. 5. Verify cookie attributes using browser developer tools.
Timeline	1 Day
Effort	Low
Evidence	This issue was verified using Browser Developer Tools by inspecting the application cookies. The session cookie does not include the SameSite attribute, which increases the risk of cross-site request attacks.

Figure 6: Session cookie without SameSite attribute



Finding #4: HTTP Only Site (No HTTPS)

Title	HTTP Only Site (No HTTPS)
Risk Level	Medium
Technical Description	The web application is accessible over HTTP instead of HTTPS. This means that data transmitted between the client and the server is not encrypted, making it vulnerable to interception.
Business Impact	Sensitive data such as login credentials, session cookies, and user input can be intercepted by attackers using techniques like man-in-the-middle attacks. This can lead to data theft, account compromise, and loss of user trust.
Remediation Steps	1. Enable HTTPS using an SSL/TLS certificate 2. Redirect all HTTP traffic to HTTPS 3. Configure secure headers such as HSTS 4. Regularly renew and maintain certificates
Timeline	1–2 Days
Effort	Low
Evidence	This issue was identified using OWASP ZAP during passive scanning. The application endpoint was observed to be accessible over HTTP without automatic redirection to HTTPS, confirming the lack of transport layer encryption.

Figure 7: OWASP ZAP alert showing HTTP-only site without HTTPS enforcement

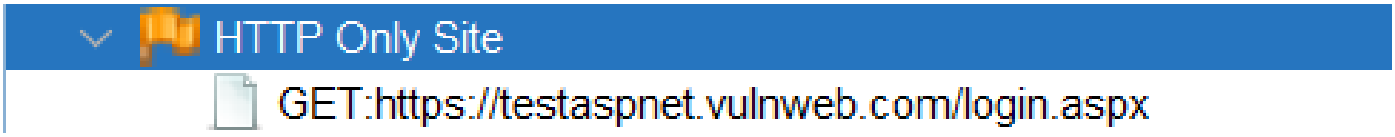
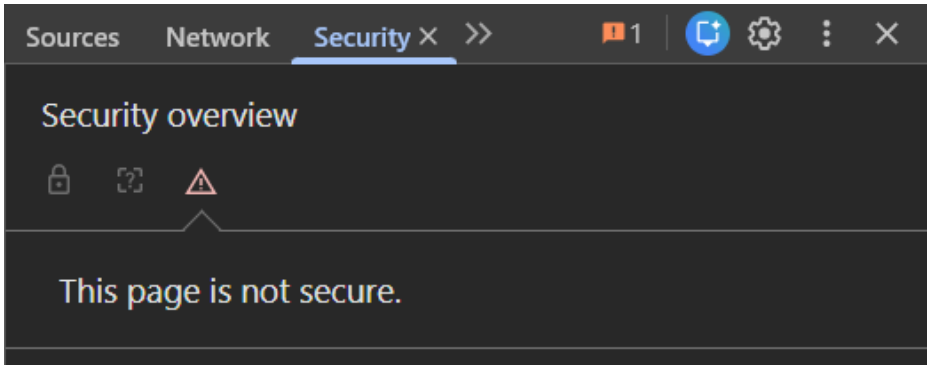


Figure 8: Browser security warning indicating that the page is not secure due to lack of HTTPS encryption



Conclusion

The vulnerability assessment identified three key issues that, while not critical, could be exploited if left unaddressed. Implementing the recommended remediation steps will significantly enhance the security posture of the web applications.

By adding proper security headers, enforcing CSRF protection, and securing cookies, the websites will be better protected against common web-based attacks. Continuous monitoring, regular security testing, and adherence to secure coding practices are recommended to maintain a strong security baseline.

The findings from both network-level and application-level analysis demonstrate that the system has a moderate security posture but requires improvements in key areas. Implementing HTTPS, strengthening session security, and enforcing proper request validation will significantly reduce the risk of exploitation.

Regular vulnerability assessments, continuous monitoring, and adherence to security best practices are essential to maintain a secure and reliable application environment over time.